

Terraform-Managed-Azure-Subscription-baseline

Problem Statement

Managing cloud infrastructure manually is error-prone, inconsistent, and difficult to scale. Different environments may be configured differently, security settings can be missed, and infrastructure changes become hard to track.

This project solves these challenges by implementing an **Infrastructure as Code (IaC)** approach using **Terraform and Microsoft Azure**, enabling automated, repeatable, secure, and scalable cloud deployments.

Solution Overview

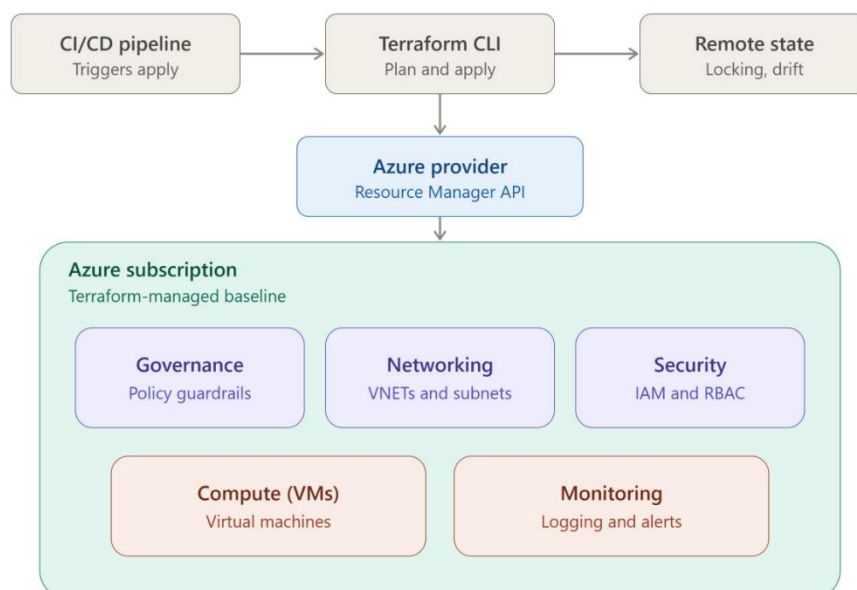
The proposed solution uses Terraform to define Azure infrastructure in code. A CI/CD pipeline automatically deploys and updates cloud resources whenever infrastructure code changes.

The architecture ensures:

- Consistent environment provisioning
- Automated infrastructure deployment
- Centralised governance and security
- Version-controlled infrastructure
- Continuous monitoring and compliance

Architecture Overview

The system consists of five major layers:



Component Explanation

1. CI/CD Pipeline

Purpose

Automates deployment of infrastructure changes.

How It Works

Whenever infrastructure code is pushed to the repository:

1. Pipeline is triggered
2. Terraform validation runs
3. Terraform plan is generated
4. Terraform apply deploys infrastructure

Benefits

- ✓ Eliminates manual deployment
- ✓ Faster infrastructure provisioning
- ✓ Reduces human error
- ✓ Supports DevOps practices

2. Terraform CLI

Purpose

Acts as the Infrastructure as Code engine.

Responsibilities

- Reads Terraform configuration files
- Creates execution plans
- Provisions Azure resources
- Updates existing infrastructure

Example

Instead of manually creating a Virtual Network:

Terraform

```
resource "azurerm_virtual_network" "main" {  
  name = "hackathon-vnet"
```

}

Show more lines

Terraform automatically creates it.

Benefits

- ✓ Infrastructure becomes reproducible
- ✓ Easy version control
- ✓ Auditability
- ✓ Environment consistency

3. Remote State Management

Purpose

Stores the current state of deployed infrastructure.

Why It Is Important

Terraform must know:

- What resources already exist
- What needs to be created
- What needs updating
- What needs deletion

Additional Features

State Locking

Prevents concurrent modifications by multiple users.

Drift Detection

Identifies differences between deployed infrastructure and Terraform code.

Benefits

- ✓ Collaboration support
- ✓ Prevents conflicts
- ✓ Infrastructure visibility

4. Azure Provider

Purpose

Acts as a bridge between Terraform and Azure.

Responsibilities

Converts Terraform instructions into Azure Resource Manager API calls.

Example:

Plain Text

Terraform Resource



Azure Provider



Azure ARM API



Azure Resource Creation

Show more lines

Benefits

- ✓ Seamless Azure integration
- ✓ Automated resource provisioning
- ✓ Consistent deployments

Azure Subscription Layer

This is where actual cloud resources are deployed.

Governance Layer

Purpose

Ensures all deployed resources follow organizational standards.

Controls

- Naming conventions
- Resource tagging
- Compliance policies
- Cost management rules

Example

A policy may block deployment of resources in unauthorized regions.

Benefits

- ✓ Compliance
- ✓ Cost control
- ✓ Standardization

Networking Layer

Purpose

Provides communication between resources.

Components

- Virtual Networks (VNets)
- Subnets
- Routing
- Network Security Rules

Benefits

- ✓ Network isolation
- ✓ Secure communication
- ✓ Scalability

Security Layer

Purpose

Controls access to cloud resources.

Components

IAM (Identity and Access Management)

Defines who can access resources.

RBAC (Role-Based Access Control)

Defines what actions users can perform.

Example

Plain Text

Admin → Full Access

Developer → Deploy Resources

Viewer → Read-only

Show more lines

Benefits

- ✓ Least privilege access
- ✓ Improved security posture
- ✓ Controlled resource management

Compute Layer

Purpose

Provides processing power.

Components

- Azure Virtual Machines
- Application workloads
- Services

Role in Architecture

Hosts applications and workloads required by business operations.

Benefits

- ✓ On-demand scalability
- ✓ Flexible deployment options
- ✓ Cloud-native operations

Monitoring Layer

Purpose

Provides operational visibility.

Features

- Logs collection
- Metrics monitoring
- Health checks
- Alert generation

Example

If CPU usage exceeds 80%, an alert is triggered.

Benefits

- ✓ Early issue detection
- ✓ Reduced downtime
- ✓ Improved reliability

End-to-End Workflow

Step 1

Developer updates Terraform code.



Step 2

Code is pushed to Git repository.



Step 3

CI/CD pipeline automatically starts.



Step 4

Terraform validates infrastructure definitions.



Step 5

Terraform plan identifies required changes.



Step 6

State file is checked and locked.



Step 7

Terraform provider communicates with Azure APIs.



Step 8

Azure resources are created or updated.



Step 9

Monitoring and logging become active.



Step 10

Secure and compliant infrastructure is available for use.